



**FUNDAMENTAL OF DIGITAL SYSTEM FINAL PROJECT REPORT
DEPARTMENT OF ELECTRICAL ENGINEERING
UNIVERSITAS INDONESIA**

16-CHANNEL PARALLEL SORTING NETWORK (BITONIC SORTER)

GROUP 10

Haidar Rafif Radithya	2406408836
Raja Avicenna Al-Kindi V.	2406416352
Clementheo Benaya R. S.	2406413810
Muhammad Zaki Al Khairi	2406432375

PREFACE

Puji dan syukur ke hadirat Tuhan Yang Maha Esa atas berkah dan karunianya sehingga laporan proyek akhir Perancangan Sistem Digital dengan judul “**16-CHANNEL PARALLEL SORTING NETWORK (BITONIC SORTER)**” dapat diselesaikan dengan baik. Besar ucapan terima kasih kepada asisten laboratorium, serta teman-teman yang telah berkontribusi dalam proses penyusunan laporan proyek akhir ini.

Laporan ini disusun untuk melengkapi proyek akhir yang merupakan pemenuhan dari mata kuliah Perancangan Sistem Digital Tahun Ajaran 2025/2026. Laporan ini membahas tentang detail dari proyek yang telah kami buat. Laporan ini meliputi latar belakang, dekripsi, hasil dan analisis dari proyek yang telah kami buat.

Adapun karena keterbatasan pengetahuan dan pengalaman kami, kami menyadari bahwa terdapat kekurangan dalam pengerjaan dan penyusunan laporan proyek akhir ini. Oleh karena itu, kritik dan saran sangat kami harapkan sehingga dapat dijadikan bahan evaluasi bagi kami kedepannya.

Depok, 7 Desember, 2025

Group 10

TABLE OF CONTENTS

CHAPTER 1.....	4
INTRODUCTION.....	4
1.1 BACKGROUND.....	4
1.2 PROJECT DESCRIPTION.....	5
1.3 OBJECTIVES.....	6
1.4 ROLES AND RESPONSIBILITIES.....	6
CHAPTER 2.....	8
IMPLEMENTATION.....	8
2.1 EQUIPMENT.....	8
2.2 IMPLEMENTATION.....	8
CHAPTER 3.....	10
TESTING AND ANALYSIS.....	10
3.1 TESTING.....	10
3.2 RESULT.....	11
3.3 ANALYSIS.....	14
CHAPTER 4.....	14
CONCLUSION.....	14

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Perkembangan pesat dalam teknologi sistem tertanam (*embedded systems*) dan *Internet of Things* (IoT) telah mendorong kebutuhan akan pemrosesan data berkecepatan tinggi, terutama pada aplikasi kritis seperti sistem radar, navigasi kendaraan otonom, dan instrumentasi medis. Dalam sistem-sistem tersebut, aliran data mentah masuk secara kontinu dan masif, di mana operasi pengurutan (*sorting*) menjadi tahap fundamental untuk keperluan validasi, seperti penghilangan *noise* melalui *median filtering* atau penentuan prioritas sinyal terkuat. Kinerja sistem secara keseluruhan sangat bergantung pada seberapa cepat data tersebut dapat diolah dan diurutkan agar keputusan dapat diambil secara *real-time* tanpa penundaan.

Namun, tantangan signifikan muncul ketika operasi pengurutan bervolume tinggi ini diimplementasikan pada arsitektur prosesor konvensional (CPU atau mikrokontroler) yang bekerja secara sekuensial. Algoritma pengurutan berbasis perangkat lunak memiliki keterbatasan inheren karena prosesor harus melakukan pengambilan instruksi (*fetching*) dan perbandingan data satu per satu secara berurutan. Hal ini mengakibatkan tingginya latensi pemrosesan, terutama ketika frekuensi data input meningkat drastis. Pada skenario di mana data masuk lebih cepat daripada waktu eksekusi CPU, akan terjadi penumpukan data (*bottleneck*) yang berisiko menyebabkan kegagalan sistem dalam merespons perubahan lingkungan tepat waktu, sebuah kondisi yang tidak dapat ditoleransi dalam sistem yang mengutamakan kecepatan dan akurasi.

Untuk mengatasi keterbatasan arsitektur sekuensial tersebut, diperlukan pendekatan berbasis perangkat keras yang memanfaatkan konsep pemrosesan paralel murni. Teknologi *Field-Programmable Gate Array* (FPGA) menawarkan solusi ideal melalui kemampuannya untuk mengkonfigurasi sirkuit digital secara kustom hingga tingkat gerbang logika. Oleh karena itu, proyek ini mengajukan

perancangan "16-Channel Parallel Sorting Network" menggunakan algoritma *Bitonic Sort*. Implementasi ini memungkinkan eksekusi pengurutan terhadap 16 jalur data dilakukan secara serentak pada level perangkat keras, mengubah kompleksitas waktu komputasi menjadi latensi sirkuit yang sangat rendah dan deterministik. Dengan demikian, sistem ini berfungsi sebagai akselerator perangkat keras (*hardware accelerator*) yang mampu meningkatkan *throughput* data secara signifikan dan meringankan beban komputasi prosesor utama.

1.2 PROJECT DESCRIPTION

Proyek akhir ini berjudul "16-Channel Parallel Sorting Network" yang merupakan rancangan akselerator perangkat keras berbasis FPGA untuk melakukan operasi pengurutan data secara masif dan paralel. Sistem ini didesain untuk menerima input berupa array yang terdiri dari 16 bilangan integer acak secara bersamaan, memprosesnya melalui serangkaian tahapan komparasi, dan menghasilkan keluaran berupa array yang telah terurut secara *ascending* (membesar) atau *descending* (mengecil) dalam waktu yang sangat singkat. Berbeda dengan pendekatan pengurutan konvensional pada CPU yang bekerja secara iteratif, proyek ini mengadopsi arsitektur *Sorting Network* dengan algoritma *Bitonic Sort*, di mana data mengalir melalui jaringan komparator fisik tanpa memerlukan instruksi percabangan yang kompleks, sehingga menjamin latensi yang deterministik dan *throughput* yang tinggi.

Secara arsitektural, inti dari sistem ini dibangun menggunakan blok-blok dasar *Compare-and-Swap* (CAS) yang berfungsi membandingkan dua nilai input dan menukarnya jika tidak sesuai urutan. Blok-blok CAS ini disusun secara hierarkis menggunakan pendekatan *Structural Programming* untuk membentuk jaringan interkoneksi bertingkat (*stages*). Untuk mengefisiensikan penulisan kode pada struktur yang repetitif ini, digunakan fitur *Looping Construct* berupa *Generate Statement* yang memungkinkan VHDL membangun puluhan unit komparator secara otomatis saat sintesis. Logika pertukaran data di dalam setiap unit CAS dienkapsulasi menggunakan *Procedure* untuk memastikan modularitas dan kemudahan penggunaan kembali (*reusability*) kode.

Selain datapath utama pengurutan, sistem ini dilengkapi dengan *Control Unit* berbasis *Finite State Machine* (FSM) yang bertugas mengatur sinkronisasi aliran data, mulai dari pengambilan input (*data latching*), proses propagasi data melalui jaringan, hingga penandaan validitas data pada output. Seluruh tipe data kustom dan konstanta sistem didefinisikan dalam sebuah *Package* terpisah untuk menjaga kerapian struktur proyek. Validasi fungsionalitas sistem dilakukan menggunakan *Testbench* otomatis yang membangkitkan vektor uji acak dan memverifikasi kebenaran urutan output secara algoritmik, memastikan bahwa setiap elemen data telah terurut dengan benar sesuai spesifikasi perancangan.

1.3 OBJECTIVES

Tujuan dari proyek akhir ini adalah:

1. Sebagai pemenuhan nilai dalam mata kuliah Perancangan Sistem Digital.
2. Mengimplementasikan pemrograman VHDL yang bersifat solutif.
3. Mendemonstrasikan penggunaan Looping Construct untuk menyusun ratusan gerbang logika secara otomatis dan efisien tanpa menulis ulang kode manual.
4. Menghasilkan sistem dengan waktu eksekusi yang tetap dan pasti, yang merupakan syarat utama dalam sistem digital real-time berkinerja tinggi.

1.4 ROLES AND RESPONSIBILITIES

The roles and responsibilities assigned to the group members are as follows:

Roles	Responsibilities	Person
Role 1	Penyusunan makalah, PPT, dan pemrograman VHDL	Clementheo Benaya R. S
Role 2	Penyusunan makalah, PPT, dan pemrograman VHDL	Haidar Rafif Radithya
Role 3	Penyusunan makalah, PPT, dan pemrograman VHDL	Raja Avicenna Al-Kindi V.

Role 4	Penyusunan makalah, PPT, dan pemrograman VHDL	Muhammad Zaki Al Khairi
--------	--	-------------------------

Table 1. Roles and Responsibilities

CHAPTER 2

IMPLEMENTATION

2.1 EQUIPMENT

Tools yang kami gunakan dalam mengerjakan proyek ini, yaitu:

- Visual Studio Code
- ModelSim
- Quartus
- Github

2.2 IMPLEMENTATION

1. Dataflow

Pada bagian dataflow, sistem diorganisasikan agar data dapat mengalir secara paralel melalui jaringan sorting. Dalam proyek ini, pendekatan dataflow banyak digunakan dalam entity `cas_unit`. Sinyal input langsung dipetakan ke output melalui logika komparasi tanpa memerlukan clock. Hal ini memaksimalkan throughput FPGA dan memungkinkan pemrosesan 16 angka secara simultan dalam satu siklus aliran data.

2. Behavioral

Modul behavioral digunakan untuk mendeskripsikan perilaku sistem pada tingkat algoritma yang lebih abstrak. Pendekatan behavioral diterapkan pada `top_sorter` untuk menangani logika register pipeline dan penghitung delay. Logika aritmatika di dalam fungsi `get_min` dan `get_max` pada `cas_unit` juga dideskripsikan secara behavioral menggunakan blok `if-else` untuk menentukan nilai mana yang lebih besar, tanpa mempedulikan gerbang logika spesifik yang akan dibentuk

3. Testbench

Proyek ini mengimplementasikan self-checking testbench. Testbench ini tidak hanya memberikan stimulus berupa array acak, tetapi juga secara otomatis memverifikasi output menggunakan statement `assert`. Testbench akan melakukan looping pada data

output untuk memastikan bahwa $\text{output}(i) \leq \text{output}(i + 1)$. Jika urutan salah, sistem akan otomatis melaporkan “SORTING FAILED” pada konsol simulasi.

4. Structural

Pada file `bitonic_net.vhd`, kita menghubungkan puluhan instansi komponen `cas_unit` secara hierarkis. Komponen-komponen ini disusun membentuk *butterfly network* sesuai algoritma Bitonic Sort. Pendekatan ini memudahkan debugging karena kita hanya perlu memastikan satu unit CAS benar, maka seluruh jaringan yang tersusun darinya akan bekerja dengan benar.

5. Looping Construct

Looping digunakan dalam dua konteks berbeda dalam proyek ini. Pertama, kita menggunakan For Generate loop dalam `bitonic_net` untuk generate puluhan kabel dan instansi `cas_unit` secara otomatis, sehingga kita tidak perlu menulis kode secara manual yang rentan akan kesalahan. Kedua, kita juga menggunakan for loop konvensional dalam `sorter_pkg` di fungsi `is_sorted` dan di dalam testbench untuk melakukan iterasi pengecekan elemen array satu per satu.

6. Procedure, Function, dan Impure Function

Pure Function: Digunakan pada `cas_unit` untuk `get_min` dan `get_max` untuk logika komparasi yang deterministik

Impure Function: Digunakan pada fungsi `log_array` di dalam package untuk mencetak status array ke konsol simulator, sehingga ada side effect output log.

Procedure: Digunakan untuk membungkus logika yang kompleks namun sering dipakai, seperti prosedur pencetakan hasil debug

7. Finite State Machine dan Microprogramming

Finite State Machine digunakan dalam `top_sorter` untuk mengontrol alur eksekusi sistem. FSM terdiri dari state IDLE, LOAD, SORTING, dan DONE. Proyek ini juga mengintegrasikan konsep microprogramming. Sinyal kontrol untuk datapath seperti `load_en`, `sort_en`, dan `latch_en` tidak di-hardcode dalam logika case, melainkan disimpan dalam sebuah tabel ROM mikroinstruksi. FSM hanya bertugas menunjuk alamat instruksi berdasarkan state saat ini, sementara sinyal kontrol diambil dari ROM

tersebut. Ini membuat unit kontrol sangat terstruktur dan mudah dimodifikasi.

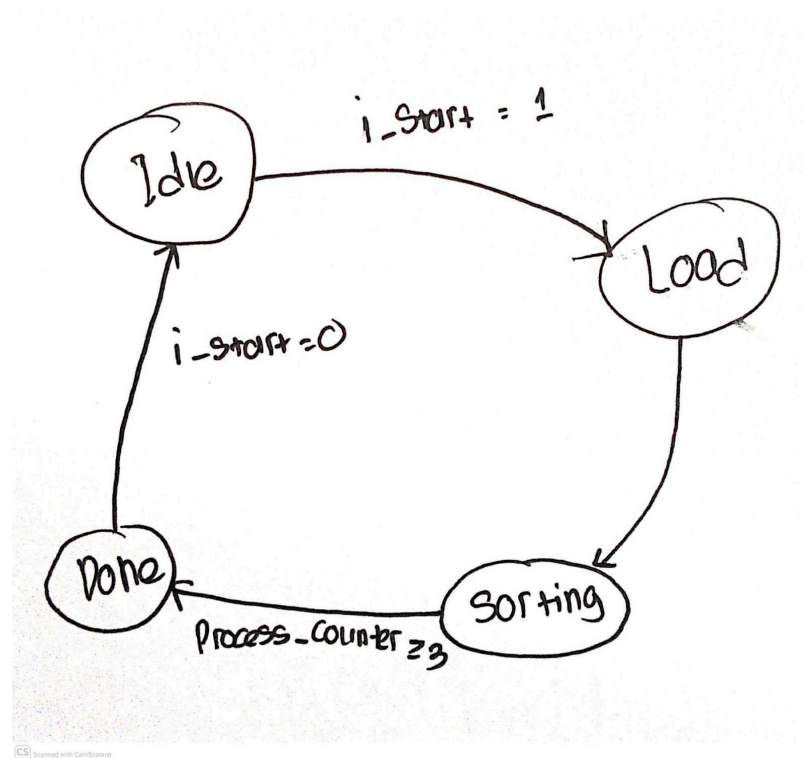


Fig 1. FSM top_sorter

CHAPTER 3

TESTING AND ANALYSIS

3.1 TESTING

Tahap testing kami lakukan untuk memastikan sorting 16-channel parallel bitonic sorter bisa berjalan sesuai dengan rencana. Kita menguji menggunakan file testbench yang sudah dibuat. Untuk melakukan testing, kita membuat banyak bentuk input yang polanya berbeda, ada data acak bernilai positif dan negatif, data yang sudah terurut, data dengan urutan terbalik, dan juga ada data yang semua nilainya sama. Sistem ini nantinya akan menjalankan proses sorting melalui beberapa tahap yang diatur oleh *Finite State Machine* (FSM) di dalam file ``top_sorter.vhd``.

Proses testing diawali dengan memberikan sinyal reset terlebih dahulu untuk menginisialisasi sistem, lalu clock signal digenerate dengan periode 10 nanosecond untuk sinkronisasi operasi. Setelah itu, testbench memberikan stimulus berupa array 16 elemen dengan tipe SIGNED 16-bit ke port input ``i_raw_data`` dan memulai proses sorting dengan cara memberikan pulse pada sinyal ``i_start``. FSM di dalam ``top_sorter.vhd`` akan masuk ke state IDLE, lalu pindah ke state LOAD untuk memuat data dari input ke register internal, selanjutnya ke state PROCESS dimana data diproses melalui jaringan bitonic sorting yang terdiri dari 10 stage kombinasional. Setelah menunggu beberapa clock cycle untuk memastikan sinyal sudah stabil melewati seluruh jaringan, FSM akan masuk ke state DONE dan mengeluarkan output sorting pada port ``o_sorted_data`` sambil mengaktifkan sinyal ``o_done``.

Untuk melakukan verifikasi bahwa proses sorting sudah berjalan dengan benar, testbench akan membaca hasil output dari port ``o_sorted_data`` dan melakukan pengecekan secara otomatis dengan loop assertion. Di testing ini, kami mengecek apakah setiap elemen di posisi i lebih kecil atau sama dengan elemen di posisi $i+1$, yang artinya array sudah terurut secara *ascending*. Pengujiannya menggunakan statement ASSERT yang akan melakukan report "SORTING FAILED" jika ada pasangan elemen yang tidak memenuhi kondisi urutan, lengkap dengan informasi indeks dan nilainya yang bermasalah. Jika semua pasangan elemen memenuhi kondisi sorting, testbench akan mereport "TEST PASSED".

Tujuan dari testing ini Adalah untuk memastikan bahwa semua komponen sistem seperti `cas\_unit.vhd` yang melakukan compare and swap, jaringan bitonic di `bitonic\_net.vhd` yang mengatur topologi sorting 10-stage, dan juga controller toplevel di `top\_sorter.vhd` yang manage datapath dan FSM yang semuanya bekerja dengan baik dan menghasilkan output yang sesuai. Tujuan kami adalah untuk semua pola input yang diberikan dari data acak, data terurut, data terbalik, dan data identik, sistem selalu bisa menghasilkan output yang sudah terurut secara *ascending* tanpa ada error dalam pengolahan data, yang menunjukkan bahwa implementasi algoritma bitonic sort kita sudah berfungsi dengan baik dan sesuai dengan perencanaan awal.

3.2 RESULT

Hasil simulasi dengan Modelsim menunjukkan bahwa kelima test case berhasil dengan tanda "TEST PASSED". Test 1 berupa data random menghasilkan output yang sudah terurut, Test 2 yang berisi data telah terurut terbukti bisa mempertahankan urutan dengan benar, Test 3 yang diurutkan terbaik berhasil dibalik, Test 4 yang identik nilainya tetap stabil, dan Test 5 yang jaraknya jauh terbukti bisa menangani nilai yang ekstrim dengan baik. Total waktu eksekusi sekitar 585ns untuk menyelesaikan seluruh pengujian.

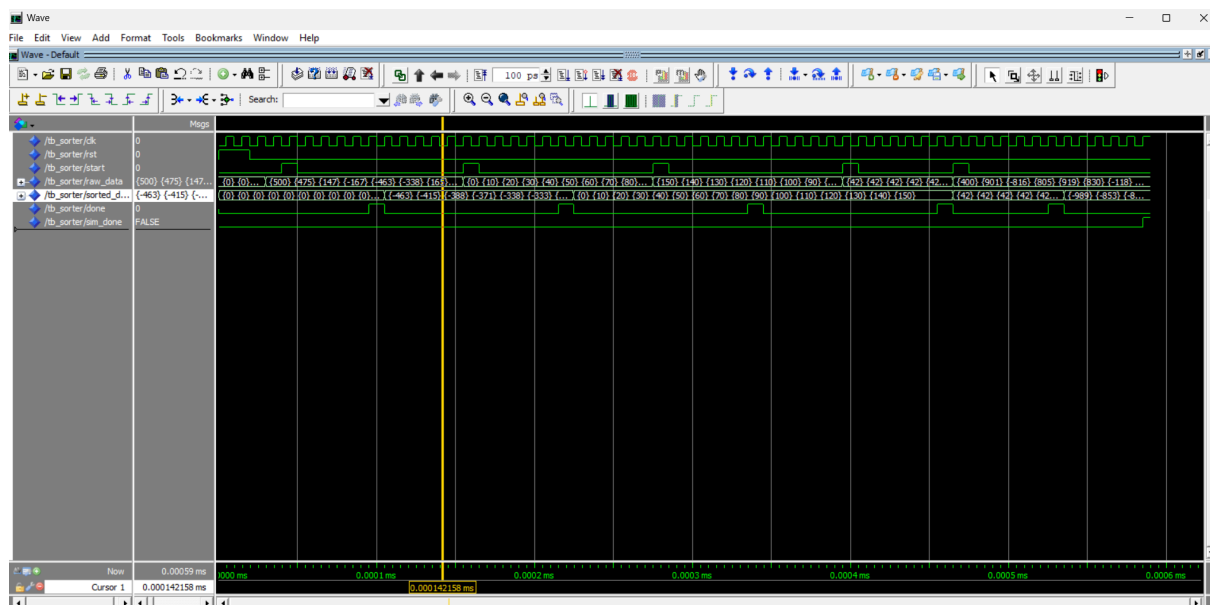


Fig 2. Testing Result

```

# Time: 105 ns Iteration: 0 Instance: /tb_sorter
# ** Note: *** TEST 1 PASSED ***
# Time: 105 ns Iteration: 0 Instance: /tb_sorter
# ** Note: =====
# Time: 155 ns Iteration: 0 Instance: /tb_sorter
# ** Note: Test 2: Testing with already sorted data...
# Time: 155 ns Iteration: 0 Instance: /tb_sorter
# ** Note: *** TEST 2 PASSED ***
# Time: 225 ns Iteration: 0 Instance: /tb_sorter
# ** Note: =====
# Time: 275 ns Iteration: 0 Instance: /tb_sorter
# ** Note: Test 3: Testing with reverse sorted data...
# Time: 275 ns Iteration: 0 Instance: /tb_sorter
# ** Note: *** TEST 3 PASSED ***
# Time: 345 ns Iteration: 0 Instance: /tb_sorter
# ** Note: =====
# Time: 395 ns Iteration: 0 Instance: /tb_sorter
# ** Note: Test 4: Testing with all identical values...
# Time: 395 ns Iteration: 0 Instance: /tb_sorter
# ** Note: *** TEST 4 PASSED ***
# Time: 465 ns Iteration: 0 Instance: /tb_sorter
# ** Note: =====
# Time: 465 ns Iteration: 0 Instance: /tb_sorter
# ** Note: Test 5: Testing with negative numbers...
# Time: 465 ns Iteration: 0 Instance: /tb_sorter
# ** Note: *** TEST 5 PASSED ***
# Time: 535 ns Iteration: 0 Instance: /tb_sorter
# ** Note: =====
# Time: 585 ns Iteration: 0 Instance: /tb_sorter
# ** Note: ALL TESTS COMPLETED
# Time: 585 ns Iteration: 0 Instance: /tb_sorter
# ** Note: =====
# Time: 585 ns Iteration: 0 Instance: /tb_sorter

```

Fig 3. Transcript Result

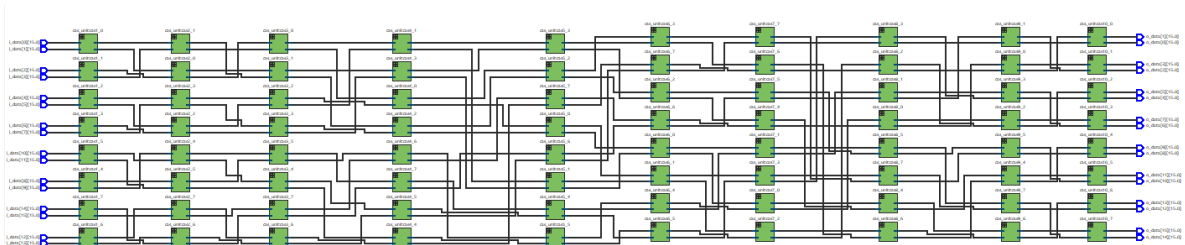
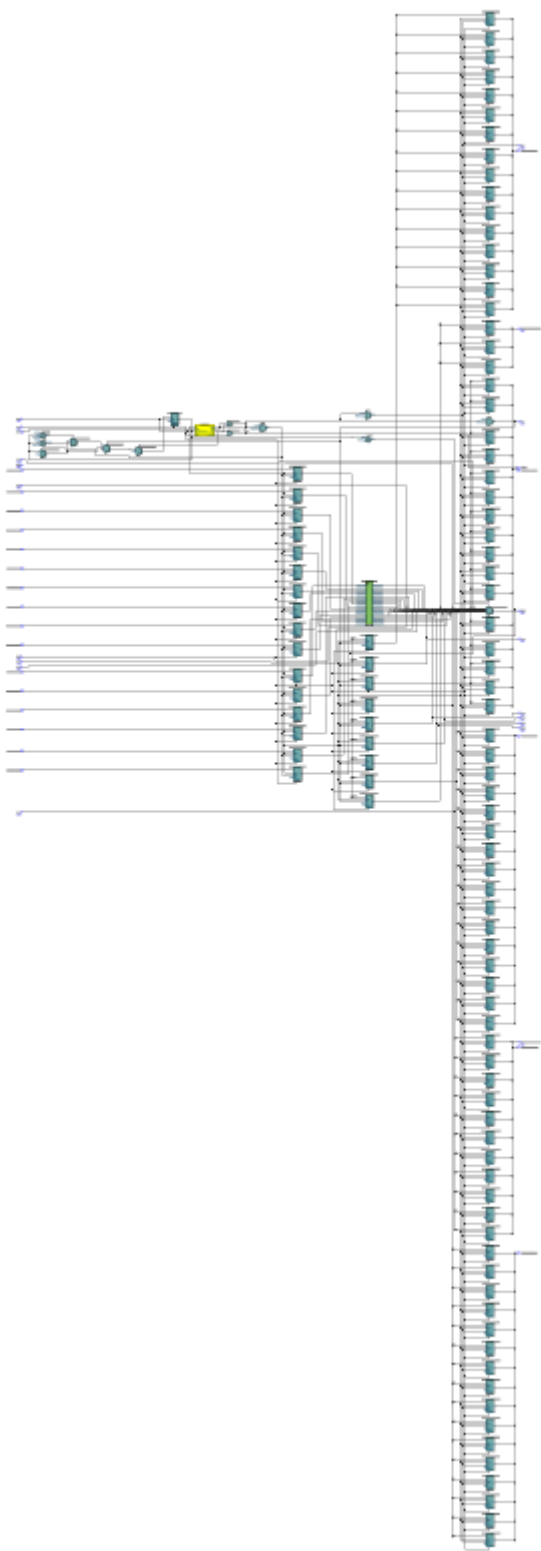
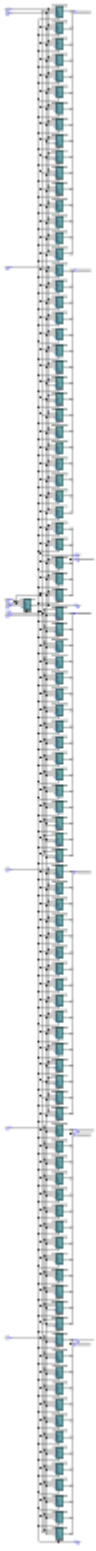


Fig 4. Synthesized Result (bitonic_net:bitonic_inst)



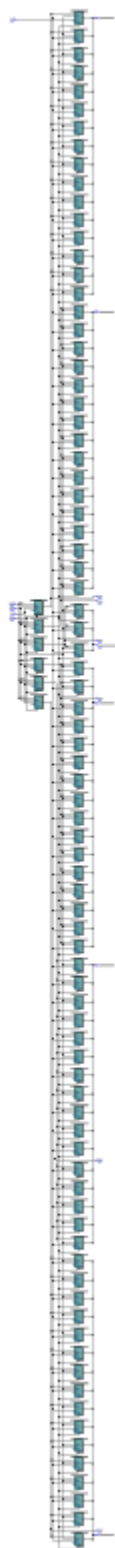


Fig 5. Synthesized

Sistem bitonic sorter terbukti konsisten dan sesuai dalam menangani berbagai kasus yang berbeda tanpa error. FSM state transitions (IDLE→LOAD→SORTING→DONE) berjalan sesuai dengan spesifikasi dengan latency 3 clock cycles untuk kestabilan dari propagasi kombinasional melalui 10 stage sorting network. Hasil yang didapat mengonfirmasi bahwa implementasi Batcher's bitonic sort algorithm dengan alternating ascending/descending sequences sudah benar secara keseluruhan.

3.3 ANALYSIS

Alur berjalannya dimulai dari FSM di `'top_sorter.vhd'` yang bertugas untuk mengatur transisi state, IDLE menerima input, LOAD memasukkan data ke register, lalu SORTING menunggu propagasi melalui `'bitonic_net.vhd'` (10 stage kombinasional), dan akhirnya DONE melatch hasil ke output. Microprogramming dengan control ROM berperan sebagai pengatur sinyal `'load_en'`, `'sort_en'`, dan `'latch_en'` sesuai state FSM. Counter `'process_counter'` diset ke threshold 3 cycles untuk memastikan sinyal stabil sebelum latch, mengakomodasi worst-case propagation delay dari 80 `cas_unit` instances.

Debugging awal menemukan bug di topologi bitonic network stage 1-6 dimana arah ascending/descending tidak sesuai algoritma Batcher. Perbaikan dilakukan dengan swap output ports pada `cas_unit` untuk grup descending, sehingga stage 1 membuat bitonic pairs (asc-desc alternating), stage 2-3 membentuk bitonic-4 groups, stage 4-6 membentuk bitonic-8 groups, dan stage 7-10 melakukan final merge ke full ascending. Setelah koreksi topologi, sistem berhasil menangani semua test case termasuk data sudah terurut yang sebelumnya gagal.

CHAPTER 4

CONCLUSION

Final Proyek (Finpro) kami yang berjudul “**16-CHANNEL PARALLEL SORTING NETWORK (BITONIC SORTER)**” berhasil kami rancang dan diimplementasikan dengan akselerator hardware yang menggunakan Intel FPGA untuk membuat ulang program atau mengonfigurasi sirkuit internal (hardware) dari perangkat yang kita buat. Kecepatan kinerja sekuensial FPGA melampaui CPU konvensional. Algoritma sort bitonic digunakan dalam arsitektur butterfly network di sistem ini. Ini memungkinkan sistem untuk memproses pengurutan secara paralel dari 16 input data, serta pengurutan pada tingkat hardware (perangkat keras), sehingga pengurutan data dapat dilakukan secara paralel. Jenis programming struktural yang digunakan pada VHDL digunakan dalam desain Sistem Sirkuit Digital ini. Ini menggabungkan sirkuit yang lebih sederhana untuk membuat sirkuit yang jauh lebih kompleks, dan unit unit dasar (Compare-and-Swap) dirancang untuk membuat jaringan interkoneksi 10 tingkat. Selain itu, kami memanfaatkan *library* “*sorter_pkg*” untuk mendefinisikan tipe data “*t_data_array*” yang merupakan konstanta dan fungsi untuk logging.

Sistem sorting yang kami buat menggunakan integrasi control unit yang memanfaatkan Finite State Machine (FSM) yang juga dipadukan dengan microprogramming pada ROM internal. Mekanisme *control unit* yang kami rancang memiliki sinkronisasi transisi state yang stabil, seperti pengambilan data (LOAD), propagasi sinyal melalui sorting (SORTING), dan Validasi hasil (DONE). Fungsi dari sistem divalidasi melalui entity “*tb_sorter*” yang merupakan testbench otomatis dan melakukan tes untuk setiap data (data acak, terurut namun terbalik, dan data yang identik, serta data negatif). Dari hasil Run -All pada testbench, didapatkan status “TEST PASSED” yang berarti program kami berhasil mengimplementasikan bitonic sort pada tingkat hardware tanpa ada kesalahan logika. Hasil “TEST PASSED” ini membuktikan bahwa algoritma batcher yang diterapkan pada sirkuit dapat mengurutkan data yang dinamis.

Secara keseluruhan, implementasi sort ini menggunakan teknologi FPGA untuk menyelesaikan masalah logika yang membutuhkan kinerja tinggi dan membutuhkan waktu pemrosesan data yang cepat pada sinyal digital. Implementasi ini "memindahkan" beban komputasi dari prosesor ke akselerator hardware, atau perangkat keras, sehingga waktu pengeksekusian pemrosesan data sangat rendah karena pemrosesan data dilakukan secara paralel pada tingkat hardware. Ketika kita ingin memperluas cakupan program tanpa menulis

ulang kode secara manual, program yang kami integrasikan juga menggunakan construct looping, seperti generate statement. Diharapkan pengembangan lebih lanjut dari rancangan sirkuit kami dapat dilakukan pada sistem tertanam yang jauh lebih kompleks, seperti pengolahan data sensor dengan Internet of Things (IoT) dan sistem radar modern.

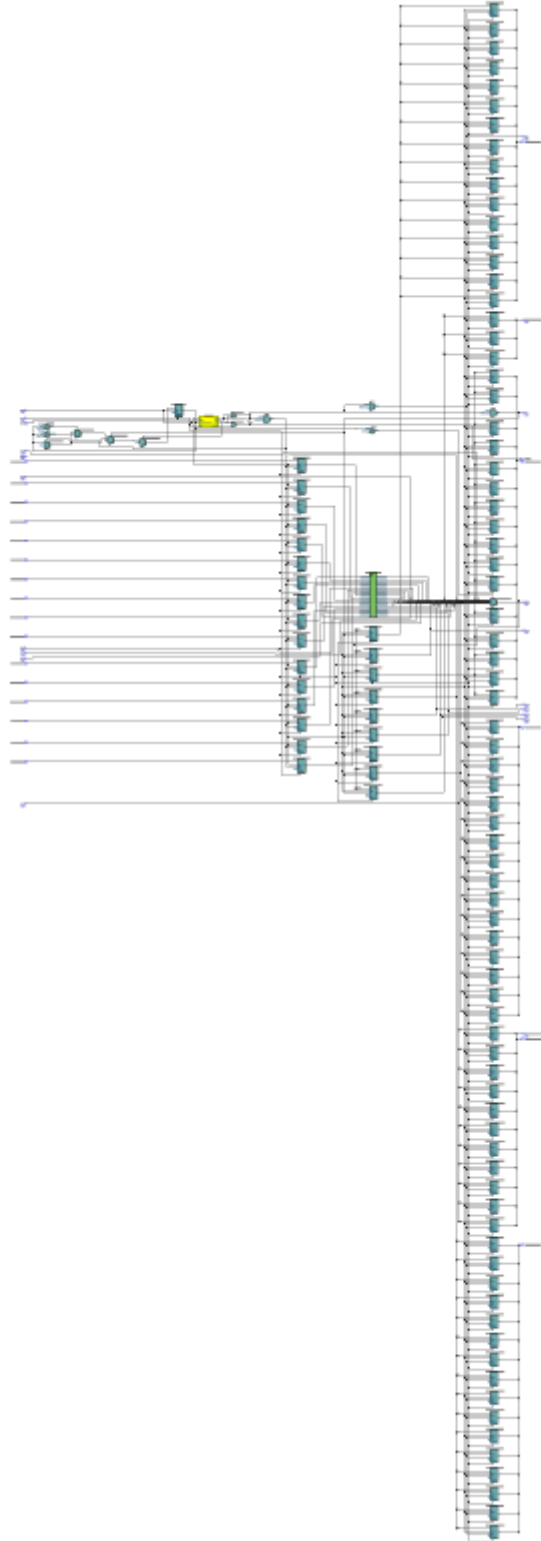
REFERENCES

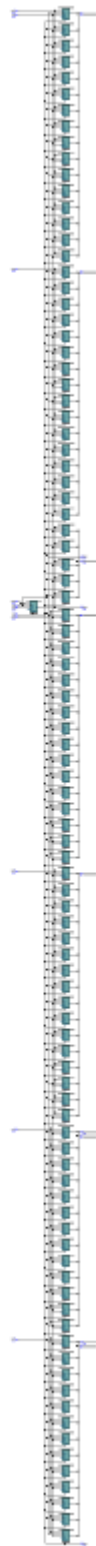
- [1] "Bitonic Sort - GeeksforGeeks," *GeeksforGeeks*, Jan. 04, 2019.
<https://www.geeksforgeeks.org/bitonic-sort/>
- [2] "FPGA Projects, Verilog Projects, VHDL Projects - FPGA4student.com."
<https://www.fpga4student.com/>
- [3] "Answers to Top FAQs," *Intel*, 2025.
<https://www.intel.com/content/www/us/en/docs/programmable/683082/13-1/advanced-synthesis-cookbook.html>
- [4] "Bitonic sorter," *Wikipedia*, Oct. 29, 2020.
https://en.wikipedia.org/wiki/Bitonic_sorter
- [5] [1]GeeksforGeeks, "Functional Programming: Pure and Impure Functions," *GeeksforGeeks*, May 15, 2023.
<https://www.geeksforgeeks.org/javascript/functional-programming-pure-and-impure-functions/>
- [6] [1]D. Williams, "Implementing a Finite State Machine in VHDL," *Allaboutcircuits.com*, Dec. 23, 2015.
<https://www.allaboutcircuits.com/technical-articles/implementing-a-finite-state-machine-in-vhdl/>
- [7] Russell, "VHDL Example Code of Generate Statement," *Nandland*, Jun. 09, 2022.
<https://nandland.com/generate/>
- [8] D. M. J. Purnomo, A. Arinaldi, D. T. Priyantini, A. Wibisono, and A. Febrian, "IMPLEMENTATION OF SERIAL AND PARALLEL BUBBLE SORT ON FPGA," *Jurnal Ilmu Komputer dan Informasi*, vol. 9, no. 2, p. 113, Jun. 2016, doi:
<https://doi.org/10.21609/jiki.v9i2.378>.
- [9] Russell, "Package File - VHDL Example," *Nandland*, Jun. 09, 2022.
<https://nandland.com/package/>
- [10] "How to create a Finite-State Machine in VHDL," *VHDLwhiz*, Aug. 25, 2018.
<https://vhdlwhiz.com/finite-state-machine/>

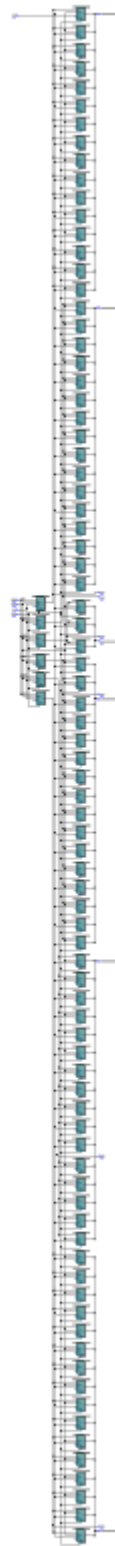
- [11] J. J. Jensen, "How to create a self-checking testbench," *VHDLwhiz*, Apr. 23, 2019.
<https://vhdlwhiz.com/how-to-create-a-self-checking-testbench/>
- [12] [1]"VHDL modules: Sorting algorithm FPGA implementations - VHDLwhiz,"
VHDLwhiz, Feb. 28, 2025.
<https://vhdlwhiz.com/product/vhdl-modules-sorting-algorithm-fpga-implementations/>
- [13] [1]J. J. Jensen, "How to use an impure function in VHDL," *VHDLwhiz*, Aug. 31, 2018. <https://vhdlwhiz.com/impure-function/>

APPENDICES

Appendix A: Project Schematic







Appendix B: Documentation

